Assignment 1 - Git & GitHub Fundamentals

Full name: Hafsa El Mahfoudy | Student ID: 158085 | Course code & section: CSC 3324 01

GitHub username: Hafsa-mhf

Repository: https://github.com/Hafsa-mhf/git-session1-Hafsa-mhf

1. Repository Setup

The supplied STARTER files were initialized with git init -b main. The .gitignore rules below exclude local.env and ordinary log files while keeping important.log available to Git.

```
local.env
*.log
!important.log
```

Troubleshooting: *.log initially hid important.log. Adding !important.log after that pattern solved the issue. Actual before/after outputs are included in the evidence folder; important.log is tracked, while local.env, app.log and debug.log are ignored.

2. README & Commit History

README.md includes the supplied identity, a project description, technologies, a Bash code block, a documentation link, and assets/project-banner.svg. The confirmed account handle is Hafsa-mhf; the required repository name is git-session1-Hafsa-mhf.

Five content commits cover setup, HTML, CSS, JavaScript, and the About feature. An explicit merge integrates the feature. Further commits record the report, account identity, online README edit, and GitHub synchronization evidence.

Commit	Change
86c89fe	Initialize starter project with README and ignore rules
92e9dfa	Clarify the project overview
d2ad1f8	Improve section spacing and borders
2e57c24	Update the project status message
813ba1c	Add the About feature on its own branch
a46ee83	Merge the About feature into main
1489f7e	Add the AI-disclosed local report and workflow evidence
9555b09	Replace the report with the updated PDF
7fffcd6	Confirm the GitHub account identity
54ed6e8	Update README directly on GitHub
712bce6	Record the actual GitHub fetch and pull evidence

3. Selective Staging

index.html and style.css were modified together. Only index.html was staged with git add index.html and committed first. style.css remained modified and was committed separately afterward.

```
Selective staging: only index.html is staged; the CSS change remains unstaged. Both diff types are
 shown.
Actual output captured in the AI-assisted execution environment.
UTC: 2026-10-12T12:02:58+00:00

$ git status --short
M  index.html
 M style.css
```

Figure 1. Selective staging: index.html is staged while style.css remains unstaged.

In short status, the first column describes the index and the second the working tree: index.html is staged; style.css is unstaged.

```
After the HTML-only commit, style.css remains modified and uncommitted.
Actual output captured in the AI-assisted execution environment.
UTC: 2026-10-12T12:02:58+00:00

$ git status --short
 M style.css

$ git diff -- style.css
diff --git a/style.css b/style.css
index b93e076..fae9c24 100644
--- a/style.css
+++ b/style.css
@@ -19,3 +19,11 @@ section {
   padding: 20px;
   border: 1px solid #e3e3e3;
 }
+
+/* Make individual sections easier to distinguish. */
+main section {
+  padding: 1rem;
+  margin-bottom: 1rem;
+  border: 1px solid #d5dce5;
+  border-radius: 6px;
+}
```

Figure 2. After the HTML-only commit, style.css remains modified. The unstaged diff shows its changes.

git diff shows unstaged changes; git diff --cached shows staged changes. Both actual diffs are preserved in evidence/04_selective_staging.txt; the remaining CSS diff is in evidence/05_selective_commit.txt.

4. Branch & Merge

git switch -c feature-about created the feature branch. The About section was added and committed there. After git switch main, git merge --no-ff feature-about integrated the work with an explicit merge commit. The feature was absent from main before merging and present afterward.

```
The About feature has been merged into main with an explicit merge commit.
Actual output captured in the AI-assisted execution environment.
UTC: 2026-10-12T12:02:58+00:00

$ git branch --list
  feature-about
* main

$ git log --graph --oneline --decorate --all
*   a46ee83 (HEAD -> main) Merge the About feature into main
|\
| * 813ba1c (feature-about) Add the About feature on its own branch
|/
* 2e57c24 Update the project status message
* d2ad1f8 Improve section spacing and borders
* 92e9dfa Clarify the project overview
* 86c89fe Initialize starter project with README and ignore rules
```

Figure 3. The feature branch and explicit merge are visible in the recorded Git history.

The merge capture was recorded before the report/evidence commit. Figures 1-4 are rendered images of actual command output from the AI-assisted environment.

Ignore-rule verification

```
Troubleshooting resolution: the exception keeps important.log visible while normal logs and local.env remain
ignored.
Actual output captured in the AI-assisted execution environment.
UTC: 2026-10-12T12:02:58+00:00

$ git status --short --ignored
?? .gitignore
?? app.js
?? assets/
?? important.log
?? index.html
?? notes.txt
?? style.css
!! app.log
!! debug.log
!! local.env

$ git check-ignore -v local.env app.log debug.log
.gitignore:1:local.env  local.env
.gitignore:2:*.log      app.log
.gitignore:2:*.log      debug.log
```

Figure 4. The ignore exception keeps important.log visible; ordinary logs and local.env are ignored. This capture predates the initial commit.

5. Remote Changes

Command	Purpose
commit	Save the staged changes in the local history.
push	Publish local commits to a remote repository.
fetch	Download remote commits and update remote-tracking references without modifying the working files.
pull	Fetch and integrate remote changes into the current branch.

An empty public repository was created, origin was connected, and main and feature-about were pushed. README was edited directly on GitHub in commit 54ed6e8. After git fetch origin, main was behind by one commit while the local branch and working files were unchanged. git pull --ff-only origin main integrated the online edit. The actual fetch/pull transcript was committed as 712bce6 and pushed.

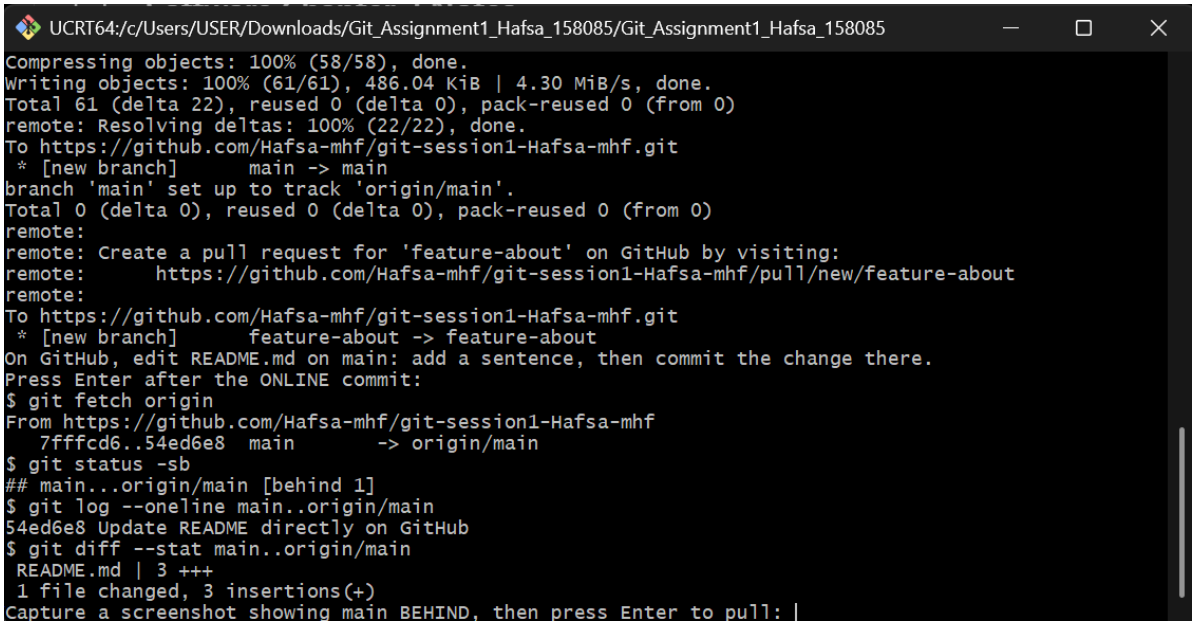


Figure 5. Student-provided Windows terminal screenshot: fetch updates origin/main and reports main behind 1 before pull.

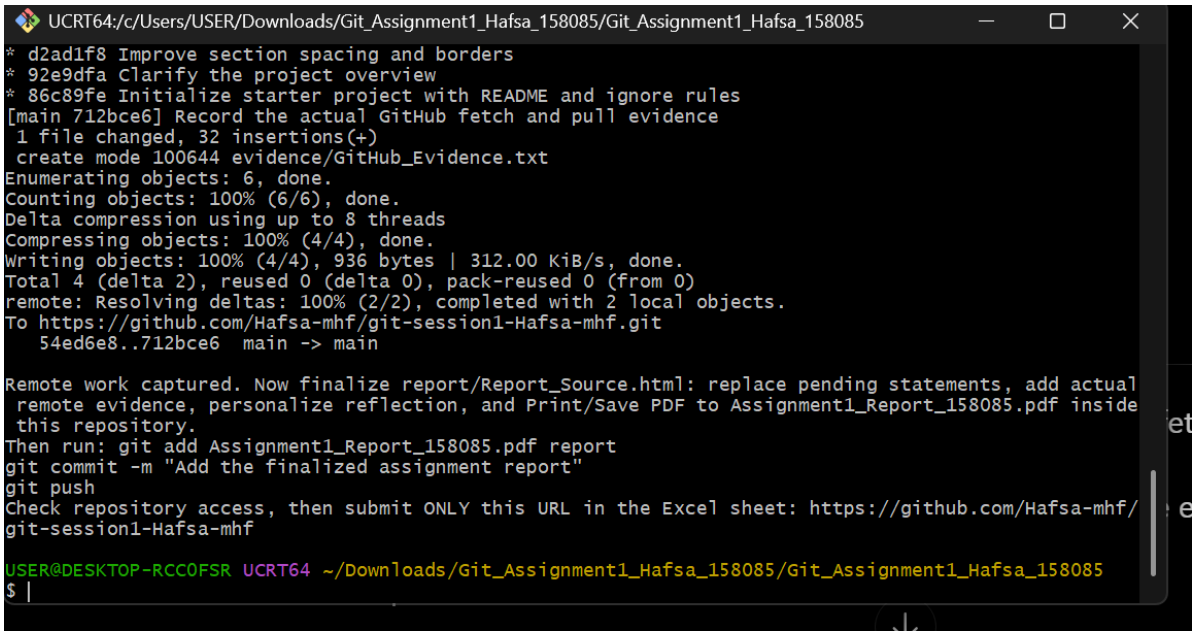


Figure 6. Student-provided Windows terminal screenshot: evidence/GitHub_Evidence.txt is committed and main is pushed successfully.

6. Fair Use of AI

ChatGPT was used to interpret the assignment, prepare the README and source edits, execute the initial local Git workflow, and compose the report after my laptop became unresponsive. My prompts requested step-by-step help and an assignment package. I used the proposed ignore rules, commit sequence, and About feature, then confirmed my username and requested changes to the report layout. The assistant checked actual status, staged and unstaged diffs, history, ignore rules, branch/merge output, and restoration from a Git bundle in its own environment. On my Windows laptop, I configured my email, replaced and committed the report, authorized Git Credential Manager, pushed the project, edited README online, and ran the guided fetch/pull exercise. I supplied terminal screenshots for review. The initial prepared commits use my name and the temporary email git-assignment@localhost; subsequent laptop commits use the email configured there.

7. Sources & Reflection

Sources: Assignment 1 and Report Template - group A.pdf; Git presentation Session 1 - group A.pdf; Assignment1_Starter - group A.zip; the recorded Git output and student-provided terminal screenshots dated October 12, 2026. AI assistance: ChatGPT, as described above.

The most difficult part of this workflow was completing Git authentication separately from signing in to GitHub in the browser. Fetch and pull also have different roles: fetch updates remote information, while pull integrates it locally. If Git behaves unexpectedly, I would first inspect git status, the current branch, and git remote -v.

Figures 1-4 document the initial AI-assisted local executions; Figures 5-6 show the subsequent workflow on my Windows laptop. The complete remote transcript is stored in evidence/GitHub_Evidence.txt in the repository.